

0816_회의

🕒 진행 상태

결정 완료

1. 데이터셋은 How2Sign만 사용

CSL-Daily 데이터셋 제외

- SignSpark 저자들이 직접 공지한 알려진 버그
 - "제공된 데이터에서 CSL-Daily의 한손(one-handed) 수어 샘플들의 회전값이 이상하게 나온다"는 걸 README에 명시해줬고, 아직 수정이 안 된 상태
- 이걸 검증 없이 섞어 쓰면 데이터 품질 문제가 학습에 그대로 섞여 들어갈 위험이 있어서, **배제하기로 결정**

2. 상대 위치 인코딩 사용

- 각자 인코딩 방식이 달랐음
- **상대 위치 인코딩으로 통일하기로 결정**
 - 이 분야에서 확립된 정답(ztta)과 가장 정확히 일치하기 때문

"목표 프레임 기준" 상대 위치, 임베딩 테이블

```
class RelativePositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=200):
        self.pos_embedding = nn.Embedding(2 * max_len + 1,
            d_model) # 학습 가능한 큰 테이블

    def forward(self, x, rel_pos):
        idx = (rel_pos + self.max_len).clamp(0, 2 * self.ma
```

```
x_len)
    return x + self.pos_embedding(idx)    # 입력 표현에 직
접 더함
```

- **기준점:** $rel_pos = \text{인덱스} - \text{목표_인덱스}$ — 목표 프레임이 항상 0이 되도록 원점을 목표로 옮김
- **주입 위치:** Transformer 인코더에 들어가기 전, 입력 토큰 표현 자체에 더함(일반적인 위치 임베딩 방식)
- **용량:** $(2 \times 41 + 1) \times 1024 \approx 84,000$ 개의 학습 가능한 파라미터(관절/시간별 풍부한 표현)
- **핵심 효과:** "이 프레임이 목표냐 아니냐"가 **항상 같은 값($rel_pos=0$)**으로 표현되니까, 우리가 겪었던 "M에 따라 같은 절대 인덱스가 목표였다가 gap이었다가" 하는 충돌이 구조적으로 원천 차단

3. MANO 시각화

- 시각화 시 MANO 모델.pkl 사용
 - MANO_RIGHT.pkl
 - MANO_LEFT.pkl
 - 슬랙에 zip 파일 존재
 - 구현은 신영 코드 참고

4. 왼손 좌표계 flip

- 은하님 코드 참고
 - 왼손·오른손을 flip 없이 그냥 같이 섞으면 왼손 시퀀스 전체(전체의 절반)가 잘못된 좌표계로 해석되고 있을 가능성이 높음
 - "LMDB에는 왼손이 진짜 SMPLX-left 좌표계로 저장되어 있는데, 모델은 오른손(WiLoR) 좌표계로 통일해서 학습"한다고 명시

```
_LEFT_HAND_FLIP_MASK = np.array([[1, -1, -1], [-1, 1, 1], [-1, 1, 1]])
```

5. 데이터셋 전처리

가림 길이

- 학습 데이터 전처리 시 가림 길이는 랜덤 샘플링 (5~30)

시퀀스 추출 방법

- 각 클립에서 stride 5로 뽑음

석우님이 전처리 코드 보내주시면 그걸로 학습하기로 통일!!

6. L2Q/L2P/NPSS 지표 통일

- train으로 학습, dev로 개인 검증, test로 최종 각 모델 간 비교
- [5, 10, 20, 30]으로 나눠서 평가
- 평가 지표
 - L2Q, L2P, NPSS
 - L2P의 경우 정확한 구현은 불가능하지만 MANO로 조금 더 정교하게 만들 수 있음

```
def mano_forward(mano_layer, hand_pose_aa, device=DEVICE):  
    T = hand_pose_aa.shape[0]  
    global_orient = torch.zeros(T, 3, device=device)  
    hand_pose = hand_pose_aa.reshape(T, -1).to(device)  
    betas = torch.zeros(T, 10, device=device)  
    output = mano_layer(global_orient=global_orient, hand_pose=hand_pose, betas=betas, return_verts=True)
```

```

    return output.joints # (T, 16, 3)

def rotation_matrix_to_quaternion_np(mats):
    quats = Rotation.from_matrix(mats).as_quat() # scipy:
    [x,y,z,w]
    return quats[:, [3, 0, 1, 2]] # [w,x,y,z]로 재배열

def l2q_error(pred_90, gt_90, n_joints=N_JOINTS):
    T = pred_90.shape[0]
    pred_mats = rotation_6d_to_matrix_np(pred_90.reshape(T
* n_joints, 6))
    gt_mats = rotation_6d_to_matrix_np(gt_90.reshape(T * n_
joints, 6))
    pred_q = rotation_matrix_to_quaternion_np(pred_mats)
    gt_q = rotation_matrix_to_quaternion_np(gt_mats)
    dist = np.minimum(
        np.linalg.norm(pred_q - gt_q, axis=-1),
        np.linalg.norm(pred_q + gt_q, axis=-1),
    )
    return float(dist.mean())

def l2p_error_mano(pred_90, gt_90, mano_layer, n_joints=N_J
OINTS):
    '''실제 MANO forward kinematics로 위치 오차 계산 (팀원들의
손수 제작 템플릿보다 정밀).'''
    T = pred_90.shape[0]
    pred_aa = sixd_sequence_to_axis_angle(pred_90.reshape
(T, n_joints, 6))
    gt_aa = sixd_sequence_to_axis_angle(gt_90.reshape(T, n_
joints, 6))
    pred_joints = mano_forward(mano_layer, pred_aa).cpu().n
umpy()
    gt_joints = mano_forward(mano_layer, gt_aa).cpu().numpy
()
    return float(np.linalg.norm(pred_joints - gt_joints, ax

```

```

is=-1).mean())

def npss(pred_seq, gt_seq):
    pred_fft = np.abs(np.fft.fft(pred_seq, axis=0)) ** 2
    gt_fft = np.abs(np.fft.fft(gt_seq, axis=0)) ** 2
    gt_norm = gt_fft / (gt_fft.sum(axis=0, keepdims=True) +
1e-8)
    pred_norm = pred_fft / (pred_fft.sum(axis=0, keepdims=T
rue) + 1e-8)
    diff = np.abs(gt_norm - pred_norm).sum(axis=0)
    weight = gt_fft.sum(axis=0)
    weight = weight / (weight.sum() + 1e-8)
    return float((diff * weight).sum())

def full_evaluate_diffusion(model, schedule, split="test",
T_values=EVAL_T_VALUES,
                                mano_layer=mano_right, batch_s
ize=16, max_batches_per_T=None,
                                sampling_steps=50):
    model.eval()
    results = {}
    ds = DiffusionSignSparkDataset(split, mode="eval")

    for T in T_values:
        T_indices = np.where(ds.T_arr == T)[0]
        sub = torch.utils.data.Subset(ds, T_indices)
        loader = DataLoader(sub, batch_size=batch_size, shu
ffle=False, collate_fn=collate_diffusion)

        l2q_list, l2p_list, npss_list = [], [], []
        for bi, batch in enumerate(tqdm(loader, desc=f"평가
T={T}", leave=False)):
            if max_batches_per_T is not None and bi >= max_
batches_per_T:
                break
            target = batch["target"].to(DEVICE)

```

```

        obs_mask = batch["obs_mask"].to(DEVICE)
        valid_mask = batch["valid_mask"].to(DEVICE)
        rel_pos = batch["rel_pos"].to(DEVICE)

        pred = sample_inbetween(model, schedule, target
t, obs_mask, rel_pos, valid_mask,
                                sampling_steps=samplin
g_steps)

        pred_np = pred.cpu().numpy()
        target_np = target.cpu().numpy()
        obs_np = obs_mask.cpu().numpy()
        valid_np = valid_mask.cpu().numpy()

        for b in range(pred_np.shape[0]):
            idx = np.where((~obs_np[b]) & valid_np[b])
[0] # gap 구간만 채점
            if len(idx) == 0:
                continue
            gt_seg, pred_seg = target_np[b, idx], pred_
np[b, idx]
            l2q_list.append(l2q_error(pred_seg, gt_se
g))
            l2p_list.append(l2p_error_mano(pred_seg, gt
_seg, mano_layer))
            npss_list.append(npss(pred_seg, gt_seg))

        results[T] = {"L2Q": np.mean(l2q_list), "L2P": np.m
ean(l2p_list), "NPSS": np.mean(npss_list),
                    "n": len(l2q_list)}
        print(f"T={T}: L2Q={results[T]['L2Q']:.4f} L2P={res
ults[T]['L2P']:.4f} "
              f"NPSS={results[T]['NPSS']:.4f} (n={results
[T]['n']})")

    return results

```

7. 프로젝트 문제

1. 문제

- How2Sign 데이터셋이 좀 다이나믹함
- context 초반과 마지막 프레임만 보다보니까 가려진 중간에 쥐었다 펴는걸 인지를 못하고 그냥 멈춰있으려는 경향 (적게 움직이려 함)
- body motion-inbetweening에서는 예측 시작 지점과 끝 지점이 몸 위치가 아예 다르다 보니까 (걸어가는 장면이라고 치면 절대 좌표 자체가 바뀜) 예측이 가능한데 손 task 같은 경우에는 손 root가 고정되어 있어서 context와 끝 프레임이 크게 다르지 않음 (중간은 막 움직이지만)

2. 해결방법 (클로드)

▼ 신영 클로드

흥미로운 관찰이야 — 그리고 실제로 이 분야에서 이미 다뤄진 문제 유형이야. 정확히 짚었어: **"쥐었다 폈다"는 시작·끝 자세가 거의 같은데 중간에만 격렬하게 움직이는 동작**은, 몸 전체 in-betweening(걷기처럼 시작·끝의 절대 위치 자체가 달라지는 경우)에는 없는 **손만의 특수한 함정**이야. 조사해볼게.

웹 검색됨

확인됐어 — 이걸 이 분야에서 ****"multimodal ambiguity(다중모드 모호성)"****라는 이름으로 이미 연구되고 있는 잘 알려진 문제야. 최신 논문(2026)이 정확히 우리 상황을 짚고 있어:

"Figure 1: **같은 시작·끝 키프레임이 주어져도, 우리 모델은 다양한 범위의 중간 동작을 생성할 수 있다**" — Generative Motion In-betweening by Diffusion (2026)

"비생성적(non-generative) 모델들은, 같은 키프레임 제약 하에서 다양한 동작 출력을 만드는 데 어려움을 겪는다" — Scalable Motion In-betweening via Diffusion (2025)

즉 우리가 겪는 문제(쥐었다 폈다를 못 잡아내고 멈춰있음)는, 우리 모델(**L1 회귀, deterministic**)이 갖는 **구조적 한계**라는 게 **문헌으로 확인**돼. 시작·끝이 비슷하면 "중간에 아무 일도 없었다"는 게 가장 안전한(loss를 낮추는) 답이라, 모델이 그쪽으로 수렴해 버리는 거야 — 우리가 전에 얘기했던 **"regression to the mean"**과 정확히 같은 현상

이고, 사용자가 관찰한 대로 손 특유의 root-고정 구조 때문에 이 문제가 몸 전체보다 훨씬 두드러지는 거야(몸은 시작-끝의 절대 위치 자체가 달라서 "아무 일도 없었다"는 답 자체가 불가능하니까).

1. 생성형(확률적) 모델로 전환 — 근본적 해법, 구현 부담 큼

Diffusion, VAE 기반 접근들이 정확히 이 문제를 풀려고 나온 거야. "하나의 정답"을 회귀하는 대신 가능한 중간 동작들의 분포에서 샘플링하니까, "정지"만이 아니라 "쥐었다 폼다"도 하나의 그럴듯한 샘플로 나올 수 있어. 다만 우리 SILK 백본을 diffusion으로 통째로 바꾸는 건 스코프가 크게 늘어나.

2. 의미 조건(semantic conditioning) 부여 — 우리에게 특히 유리한 해법

CMIB 같은 논문들이 "제스처 타입 같은 의미 정보로 조건을 건다"는 걸 이미 다뤘잖아. 우리는 이 조건을 이미 데이터에 갖고 있어 — SignSpark의 `translation` (번역문)이나 `gloss` (수어 단어)! "지금 무슨 수어를 하고 있는지"라는 정보를 모델에 추가로 주면, 모델이 "이 수어는 원래 중간에 손을 쥐는 동작이 들어간다"는 걸 텍스트에서 힌트받아 예측할 수 있어. 이건 몸 동작 in-betweening 논문들엔 없는, 우리 데이터만의 특권이야.

3. SignSpark 원래의 다중 키프레임 구조를 되살리기 — 가장 직접적인 원인 해결책

이게 사실 제일 근본적일 수 있어. 아까 원본 소스(`pose_datasets_lmdb.py`)에서 확인했잖아 — 공식 SignSpark는 "각 수어 세그먼트마다 시작/중간/끝" 키프레임을 준다고. 우리가 SILK 구조에 맞추려고 이걸 "컨텍스트+목표 1개"로 단순화하면서, 정확히 "쥐는 동작의 정점(peak)"에 해당하는 중간 키프레임 정보를 버린 셈이야. 우리 윈도우가 하필 그 "쥐었다 펴는" 동작 전체를 gap 안에 통째로 담고, 그 정점 키프레임을 놓치고 있었을 가능성이 높아 — 이게 사용자가 관찰한 문제의 진짜 원인일 수 있어.

해결: gap 구간 안에 `segment` 라벨 기준으로 "중간 키프레임"이 있으면, 그것도 컨텍스트처럼 추가로 노출시키는 방식으로 확장 — 원래 SILK(1개 목표)보다 *****목표가 여러 개*****인 구조로 바꾸는 거야.

4. 손실 함수에 다양성(diversity) 유도 항 추가 — 중간 난이도

가장 단순하게는, "예측이 GT와 가깝다"만이 아니라 *****예측 시퀀스가 시작·끝보다 충분히 움직여야 한다*****는 페널티를 손실에 추가하는 방법. 예를 들어 예측된 시퀀스의 프레임간 변화량 총합이 너무 작으면(=너무 정적이면) 벌점을 주는 식이야. 구현이 간단하고 우리 기존 L1 손실에 추가 항으로 엮기 쉬워.

5. 속도(velocity) 기반 보조 손실 — 4번의 변형, 더 구체적

GT의 프레임간 속도(회전 변화량)와 예측의 속도를 별도로 맞추는 손실을 추가. 이러면 "위치는 대충 맞는데 움직임 자체가 밋밋한" 문제를 직접 겨냥할 수 있어.

8. 그래서 할 것

- 각자 다른 모델을 해보기로 함
- 신영
 - diffusion 기반 생성형 모델
- 석우
 - SILK 기반을 좋은 아이디어만 취해서 다시 학습 + 다양성 유도 항 추가, 속도 기반 보조 손실
- 은하
 - 다른 모델 + 다중 키프레임 구조

9. 타임라인

~ 8/20 (목 세션 전)

- 각자 모델 돌리고 시각화 및 평가 점수 공유

~ 8/24

- PPT 담당
 - PPT 제작
- 나머지
 - 모델 수정 및 고도화
 - 코드 정리 및 깃헙 업로드

- 시각화 자료들 정리

~ 8/24 (월)

- 팀별 KUBIG Github 업로드: KUBIG_2026_FALL > KUBIG CONTEST > 팀별 카테고리 (팀별 폴더 제작 후 해당 폴더에 업로드)
- readme.md: 프로젝트 소개글 작성
- 사용 코드
- 발표 PPT

~ 8/27 (목)

- 발표 대본 및 발표 연습